

CURSO · JORNADA 1 DE 2

Microsoft Fabric

La plataforma analítica unificada de Microsoft

4 horas · master class · caso [Aurora Energía](#)

¿Qué nos llevaremos hoy?

- Entender **qué es Fabric** y dónde encaja en el ecosistema Microsoft.
- Comprender **OneLake** y por qué es la pieza disruptiva.
- Distinguir **Lakehouse, Warehouse, Eventhouse** y **Semantic Model**.
- Construir un flujo **end-to-end**: **CSV → Lakehouse → Spark → Warehouse → Power BI**.
- Tocar **Real-Time Intelligence** con KQL.
- Salir con un **workspace funcional** sobre el que practicar en casa.

Hilo conductor: Aurora Energía — operador ficticio con red de estaciones de servicio, comercializadora eléctrica y división de logística.

Agenda · 4 h con descanso de 15 min

BLOQUE	MIN	TEMA
M0	15	Bienvenida y contexto
M1	30	Fundamentos · arquitectura · OneLake · licencias
M2	35	Lakehouse, Warehouse y SQL Endpoint
M3	40	Ingesta · Dataflow Gen2 + Pipelines
	15	Descanso
M4	35	Notebooks y Spark
M5	30	Warehouse en profundidad + Direct Lake
M6	25	Real-Time Intelligence · Eventhouse + KQL
M7	25	Power BI · modelo semántico Direct Lake
M8	10	Cierre y deberes

Antes de empezar

Todo el contenido del curso está en <https://github.com/intelequia/Curso-Microsoft-Fabric>

1. Lee [requisitos.md](#) y prepara tu equipo con todo lo indicado **antes** de la primera jornada.
2. Sigue [00-preparacion-entorno.md](#) para crear tu trial de Microsoft Fabric. Si tu organización ya dispone de capacidad Fabric, hablamos en el primer bloque sobre cómo solicitarte un workspace propio.
3. Descarga la carpeta [assets/data](#) en local: la usarás en varios ejercicios.

NOTA: para los que no les haya dado tiempo, hemos preparado un tenant auroraenergias.onmicrosoft.com, con credenciales para todos.



M1 · 30 min

Fundamentos

Arquitectura, OneLake, capacidades y licencias

¿Qué es Microsoft Fabric?

- Plataforma **SaaS** de analítica unificada · GA noviembre 2023.
- Reúne en un único producto: **ingesta, data engineering, data warehouse, real-time, data science, Power BI** y, desde 2024–2026, **Fabric Databases** y **Fabric IQ**.
- Construido sobre tres pilares:
 - **OneLake** · un único almacenamiento por tenant.
 - **Capacidades F** · una sola unidad de cómputo para todo.
 - **Experiencias por rol** · "lentes" sobre el mismo workspace.

Fabric no reemplaza a Azure: lo simplifica para casos analíticos.

OneLake · el OneDrive de los datos

- Data Lake **lógico por tenant**, multi-cloud, sobre ADLS Gen2.
- Jerarquía: Tenant > Workspace > Item > Files & Tables .
- Todo se almacena en **Delta-Parquet** (formato abierto).
- Cualquier motor que lea Delta puede leer OneLake.
- Dos primitivas clave:
 - **Shortcuts** punteros virtuales a datos externos (ADLS, S3, GCS) o internos.
 - **Mirroring** réplica casi en tiempo real desde Cosmos DB, Azure SQL, Snowflake, Fabric SQL DB.

Workloads · una plataforma, muchas lentes

EXPERIENCIA	PARA QUÉ
Data Factory	Pipelines, Dataflow Gen2, Copy Job
Data Engineering	Lakehouse, Notebooks, Spark Job Definitions
Data Warehouse	T-SQL nativo lectura/escritura
Data Science	Notebooks, MLflow, modelos
Real-Time Intelligence	Eventstream, Eventhouse, Activator
Power BI	Semantic models, reports, dashboards, apps
Databases	Fabric SQL Database
Industry Solutions	Healthcare, Sustainability, Retail

Capacidad y licencias

CONCEPTO	QUÉ ES	EQUIVALENCIA
F-SKU (F2 → F2048)	Unidad de cómputo de Fabric, facturada por hora, pausable	F64 ≈ P1 de Power BI Premium
Power BI Free	Solo consumir reportes en el área personal	—
Power BI Pro	Compartir reportes en workspaces que no estén en F64+	Incluido en M365 E5
Power BI PPU	Funcionalidades premium por usuario, sin capacidad	—
Fabric capacity	Imprescindible para crear cualquier item Fabric	Por workspace

Regla de oro: para distribuir reportes a usuarios **Free** necesitas **F64+**. Por debajo, los consumidores deben tener Pro/PPU.

Trial y roles

- **Trial Fabric:** 60 días, equivalente a una FT1 (~F64 en muchos límites). Sin tarjeta, una por usuario.
- **Roles de workspace** (heredados de Power BI, ahora aplicados a **todos** los items Fabric):
 - **Admin** · gobierna el workspace.
 - **Member** · gestiona y publica.
 - **Contributor** · crea y edita.
 - **Viewer** · sólo lectura.
- Permisos finos por item (compartir un Lakehouse en lectura sin compartir el workspace).
- **OneLake Data Access roles + RLS / CLS / OLS** se ven en M5 y en Jornada 2 con Purview.

Tres ideas que se repiten todo el curso

- **SaaS-first** · creas un workspace y empiezas. Cero infraestructura.
- **OneLake** · un almacenamiento, un formato (Delta), un permiso.
- **Una capacidad** · mismos **Capacity Units** para Spark, T-SQL, KQL y Power BI.

M2 · 35 min

Lakehouse vs Warehouse

Y el famoso SQL Endpoint

Lakehouse

- Estructura física: **Files** (cualquier formato) + **Tables** (Delta-Parquet, descubribles).
- Ingesta multi-modo: portal, OneLake File Explorer, Spark, Dataflow, Pipeline, shortcut.
- **Gratis** trae:
 - **SQL Analytics Endpoint** (sólo lectura).
 - **Default semantic model** para Power BI.
- Encaje natural: data lake moderno, ingesta cruda + curado, ML, Spark, datos no estructurados.

Warehouse

- **T-SQL nativo, lectura/escritura** · DML completo: `INSERT` , `UPDATE` , `DELETE` , `MERGE` .
- Almacenamiento debajo: **Delta-Parquet en OneLake** (mismo formato que Lakehouse).
- Soporta:
 - **Vistas, procedimientos, funciones.**
 - **Transacciones cross-table.**
 - **OLS, CLS, RLS, Dynamic Data Masking.**
- Encaje natural: equipo SQL, modelo en estrella servido a BI, lógica con DML/transacciones.

Comparativa cara a cara

CARACTERÍSTICA	LAKEHOUSE + SQL ENDPOINT	WAREHOUSE
Lenguaje principal	PySpark / Spark SQL	T-SQL
Escritura SQL	✗ (endpoint solo lectura)	✓
Datos no estructurados	✓ (carpeta Files)	✗
DML T-SQL	✗	✓
Transacciones multi-tabla	Limitado (Delta)	✓ ACID
RLS / CLS / OLS	Parcial	Completo
Dynamic Data Masking	Limitado	✓
Stored procs / funciones	✗	✓
Direct Lake en Power BI	✓	✓

¿Cuándo cada uno?

- **Lakehouse** → ingesto fuentes heterogéneas, necesito **raw** y **curated**, hago ciencia de datos, no requiero DML SQL.
- **Warehouse** → equipo SQL puro, DML/MERGE, transacciones multi-tabla, gobierno fino sobre tablas SQL.
- **Patrón habitual:**
 - **Lakehouse** para **bronze** y **silver**.
 - **Warehouse** para **gold** (modelo dimensional servido a Power BI).

En Aurora Energía: `lh_aurora` aterriza la materia prima, `wh_aurora` sirve el modelo certificado.

Demo en vivo · 10 min

1. Crear `lh_aurora` desde + **Nuevo elemento** → **Lakehouse**.
2. Subir `clientes.csv` a `Files/landing/`.
3. **Cargar a tablas** → `clientes_raw`.
4. Cambiar al **SQL Endpoint** y `SELECT TOP 10 * FROM clientes_raw;`.
5. Crear `wh_aurora` y ejecutar:

```
CREATE TABLE dim_cliente_demo (id INT, nombre VARCHAR(100));
INSERT INTO dim_cliente_demo VALUES (1, 'Demo');
```

6. Comparar con el SQL Endpoint del Lakehouse (que **no** permitiría esto).

Para llevarse a casa

- Lakehouse y Warehouse **comparten OneLake** — no son silos.
- Si dudas, empieza por **Lakehouse** y crea Warehouse cuando necesites DML.
- El **SQL endpoint** es la puerta de entrada a tu Lakehouse para todo el mundo SQL.



M3 · 40 min

Ingesta

Dataflow Gen2, Pipelines, Copy Job, Mirroring

Opciones de ingesta · cuándo usar cada una

OPCIÓN	CUÁNDO ENCAJA	LENGUAJE
Dataflow Gen2	Ingesta + transformación con Power Query, low-code, batch	M / sin código
Data Pipeline	Orquestación, control de flujo, copia masiva	UI / JSON
Copy Job	Copia incremental gestionada (CDC + watermark)	UI
Eventstream	Streaming continuo (IoT Hub, Event Hub, Kafka)	UI
Mirroring	Réplica casi en tiempo real desde Azure SQL, Cosmos, Snowflake	UI
Notebook Spark	Ingesta programática con <code>spark.read</code>	PySpark

Dataflow para transformar, Pipeline para orquestar.

Dataflow Gen2 al detalle

- Heredero del Dataflow clásico de Power BI · multi-destino, motor mejorado (**Fast Copy**).
- UX = **Power Query Online** · vista previa, [Applied Steps](#), editor M.
- Destinos: Lakehouse, Warehouse, KQL DB, Azure SQL DB, Synapse SQL, ADX.
- **Refresco** programable, identidades de servicio y service principal.
- **Plantillas .pqt** importables y exportables.
- **Fast Copy** se activa automáticamente para conectores y cargas grandes.

Data Pipeline al detalle

- Orquestador heredero de **Azure Data Factory v2**, integrado en Fabric.
- **Actividades**: Copy data, Dataflow, Notebook, Spark Job, Stored Procedure, Lookup, ForEach, If/Switch, Wait, Web, Office365, Teams.
- **Eventos** entre actividades:
 - ● éxito · ● fallo · ● omisión · ● al completar.
- **Programación**: cron, eventos (**file arrived**), manual.
- **Monitoring Hub**: vista cross-workspace de ejecuciones.

Patrón habitual · Pipeline llama a Dataflow

```
[Pipeline pl_aurora_ingesta]
├─ Dataflow → df_clientes      (carga clientes_raw)
├─ Dataflow → df_ventas       (carga ventas_raw)
├─ Notebook → nb_silver_clean (raw → silver)
├─ Stored Proc (Warehouse) → sp_load_dim_cliente
├─ On success: Office365 Outlook → correo OK
└─ On failure: Teams → mensaje al canal de soporte
```

Empieza siempre con un Pipeline madre que oriente todo el flujo: te dará trazabilidad y errores controlados.

Mirroring · ETL cero

- Workspace → + **Nuevo** → **Mirrored database**.
- Selecciona origen: Azure SQL DB, Cosmos DB, Snowflake, Fabric SQL DB.
- En segundos aparece como base **espejo** sobre OneLake en Delta.
- Cualquier item Fabric puede leerla.

En Aurora Energía: replicamos el ERP (Azure SQL) **sin construir ETL** para análisis. Coste: capacidad consumida + storage marginal.

Demo en vivo · 15 min

Parte 1 · Dataflow Gen2

1. + Nuevo → Dataflow Gen2 → `df_clientes`.
2. Get data → Text/CSV → `clientes.csv`. Tipos correctos.
3. Destino: `lh_aurora_clientes`, modo **Replace**.
4. Publish y ejecutar.

Parte 2 · Data Pipeline

1. + Nuevo → Data pipeline → `pl_aurora_ingesta`.
2. Actividad **Dataflow** → `df_clientes`.
3. Actividad **Copy data** → `productos.csv` → tabla `productos`.
4. Validar, ejecutar, abrir **Monitoring Hub**.

Mensajes clave del bloque

- **Dataflow** transforma, **Pipeline** orquesta.
- Si tu origen es **operacional**, no construyas ETL: usa **Mirroring**.
- **Monitoring Hub es tu amigo** · úsalo siempre antes de pedir ayuda.



Descanso • 15 min

Volvemos para Spark, Warehouse y Real-Time

M 4 · 3 5 min

Notebooks y Spark

La herramienta de transformación más potente

Notebooks en Fabric

- Editor estilo **Jupyter** con celdas de código y markdown.
- Lenguajes: **PySpark**, **Spark SQL**, **Scala**, **SparkR** / **sparklyr**.
- Conexión nativa a uno o varios **Lakehouses** desde el panel **Explorer**.
- Ejecución sobre **Apache Spark gestionado** · sin provisionar cluster.

En Fabric el Spark **ya está**. No esperas 5 minutos al pool: en segundos tienes sesión.

Spark gestionado · pools, environments, sessions

- **Starter Pool** · pool predeterminado, listo en segundos.
- **Custom pools** · tamaño, autoscale, librerías propias.
- **Environments** · paquetes Python/Java + configuración Spark + recursos.
Versionables y compartibles.
- **Sessions** · ejecución activa. Pueden quedar **calientes** (high concurrency) entre celdas y notebooks.

APIs que vas a usar

- **PySpark** · `spark.read.format("csv")`, `df.write.format("delta").saveAsTable(...)`.
- **Spark SQL** · celdas con `%%sql`. Ideal para perfil SQL puro.
- **NotebookUtils** (`notebookutils`) · OneLake, secretos, fs, jobs encadenados.
- **Pandas API on Spark + fabric-data-functions** · DataFrame **pandas-like** sobre Lakehouse.

Acceso a OneLake desde Spark



```
# Lectura de tabla del Lakehouse adjunto
df = spark.read.table("lh_aurora.clientes")

# Lectura de fichero en Files/
df_raw = (spark.read
          .option("header", True)
          .csv("Files/landing/ventas.csv"))

# Escritura como tabla Delta gestionada
(df_silver.write
 .mode("overwrite")
 .format("delta")
 .saveAsTable("ventas_silver"))
```

Caso Aurora · de **bronze** a **silver**

Notebook `nb_aurora_lab` que:

1. Lee `clientes`, `productos`, `estaciones`, `ventas_raw`.
2. Limpia `ventas_raw` · descarta `importe <= 0`, normaliza fechas, deriva `año`, `mes`, `día_semana`.
3. **Join** con `productos` para añadir `categoria`, `unidad_medida`.
4. Escribe `ventas_silver` como tabla Delta.

Buenas prácticas

- Cabecera **markdown** en cada notebook: objetivo, inputs, outputs.
- `display(df)` en lugar de `df.show()` · visuales interactivos.
- Habilitar **Git integration** del workspace.
- Notebooks estables → **Spark Job Definition** invocable desde Pipeline.
- Para grandes volúmenes, **particionar Delta** por columna de baja cardinalidad (ej. `año`).

Para llevarse a casa

- Spark en Fabric es **gestionado** y de pago **por uso real**.
- Notebook es la transformación más potente — **pero no la única**.
- Convierte notebooks estables en **Spark Job Definitions** y orquíetalos con Pipeline.

M5 · 30 min

Warehouse en profundidad

Y el modo **Direct Lake** de Power BI

Warehouse · refresco rápido

- T-SQL completo · DDL, DML, vistas, procs, funciones, transacciones multi-tabla.
- Almacena en **Delta-Parquet sobre OneLake** (visible desde el Lakehouse vía cross-DB).
- Soporta:
 - `CREATE TABLE AS SELECT` (CTAS) y `SELECT INTO`.
 - `MERGE` (vía pattern).
 - **Cross-database queries** a Lakehouse y a otros Warehouses.

Modelando el **gold** de Aurora

```
CREATE TABLE dim_cliente (cliente_id INT, nombre_cliente VARCHAR(150),
                             email VARCHAR(150), dominio_email VARCHAR(100),
                             segmento VARCHAR(20),
                             fecha_alta DATE);

CREATE TABLE dim_producto (producto_id INT, nombre VARCHAR(150),
                             categoria VARCHAR(50), unidad VARCHAR(20));

CREATE TABLE dim_estacion (estacion_id INT, nombre VARCHAR(150),
                             provincia VARCHAR(50), tipo VARCHAR(30));

CREATE TABLE dim_tiempo (fecha DATE, anio INT, mes INT, dia INT,
                           trimestre INT, dia_semana INT);

CREATE TABLE fact_ventas (venta_id BIGINT, fecha DATE,
                           cliente_id INT, producto_id INT, estacion_id INT,
                           cantidad DECIMAL(12,3), importe DECIMAL(12,2));
```

Carga, vistas y seguridad

```
CREATE OR ALTER PROCEDURE sp_load_dim_cliente AS
BEGIN
    TRUNCATE TABLE dim_cliente;
    INSERT INTO dim_cliente
    SELECT cliente_id, nombre_cliente, email, dominio_email, segmento, fecha_alta
    FROM lh_aurora.dbo.clientes;      -- cross-DB al Lakehouse
END;
```

- **RLS** · política basada en `USER_NAME()` o claim de Entra.
- **CLS** · `GRANT SELECT` sólo sobre columnas concretas.
- **OLS** · ocultar tablas / columnas a roles.
- **Dynamic Data Masking** · enmascarar DNI, email, teléfono.

Direct Lake · **el santo grial** del modelo semántico

- Power BI lee **directamente las tablas Delta** del Lakehouse / Warehouse.
- **Velocidad de Import + frescura de DirectQuery, sin coste de refresh.**
- **Fallback automático a DirectQuery** si la consulta excede límites.
- Requisitos:
 - Capacidad **F-SKU** (Trial vale).
 - Modelo creado desde el item Lakehouse / Warehouse.
 - Columnas tipadas correctamente.
 - Sin transformaciones complejas en el modelo.

Direct Lake on OneLake (DLOL)

- PBIP "vacío" desde Power BI Desktop apunta al Lakehouse vía DLOL.
- **Independiente del Lakehouse origen** → mejor para **multi-tenant** y para mover el modelo entre workspaces sin recrearlo.
- Encaje natural: ISVs, productos data, plantillas reutilizables.

Demo en vivo · 12 min

1. En `wh_aurora` ejecutar el DDL de las 5 tablas + `sp_load_dim_cliente`.
2. Cross-DB al Lakehouse para llenar `fact_ventas`.
3. Crear vista `vw_kpi_ventas_diarias`.
4. + **Nuevo modelo semántico** desde el Warehouse · 5 tablas + vista. Relaciones.
5. Reporte nuevo · matriz **año / categoría / SUM(importe)**. Comentar tiempos.
6. Verificar en propiedades del modelo que está en **Direct Lake**.

Mensajes clave

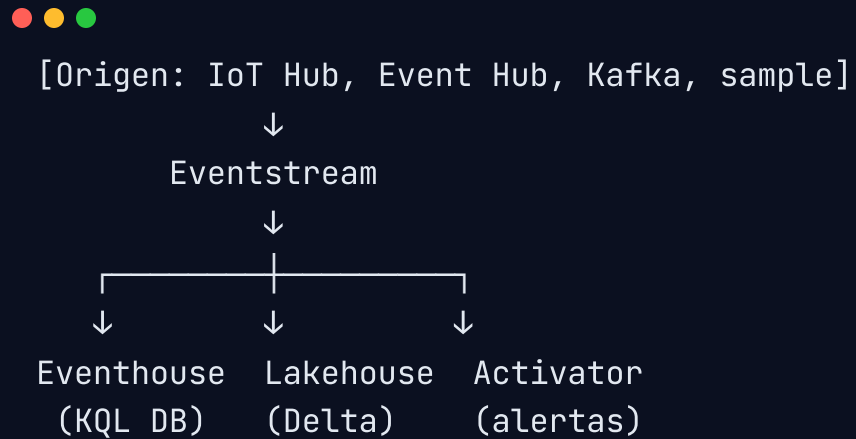
- Warehouse es la pieza para el equipo SQL clásico — pero **comparte storage** con Lakehouse.
- **Direct Lake** elimina el dilema clásico **Import vs DirectQuery**.
- Cuanto más limpio el Delta, mejor el rendimiento de Direct Lake.

M 6 · 2 5 m i n

Real-Time Intelligence

Eventstream, Eventhouse y KQL

Mapa de la Real-Time Intelligence



- **Eventstream** · ingesta no-code, transformaciones simples.
- **Eventhouse / KQL DB** · motor analítico para series temporales y logs.
- **Activator** · motor de reglas → correo, Teams, Power Automate, Pipeline.
- **Real-Time Dashboard** · estilo Grafana / Kusto Explorer integrado.

¿Eventhouse o Lakehouse?

CASO	MEJOR OPCIÓN
Telemetría IoT, logs, clickstream, security events	Eventhouse / KQL
Histórico analítico de negocio (ventas, finanzas)	Lakehouse / Warehouse
Query en milisegundos sobre miles de millones de eventos	Eventhouse
Unión a un modelo dimensional de negocio	Lakehouse / Warehouse

KQL en 5 minutos



```
// Top 10 estaciones por número de eventos en las últimas 24 h
TelemetriaSurtidor
| where Timestamp > ago(24h)
| summarize Eventos = count() by EstacionId
| top 10 by Eventos desc

// Detección de surtidores con caudal anómalo
TelemetriaSurtidor
| where Timestamp > ago(1h)
| summarize CaudalMedio = avg(Caudal)
               by EstacionId, SurtidorId, bin(Timestamp, 5m)
| where CaudalMedio < 0.5 or CaudalMedio > 80
```

Sintaxis **pipe-based** · si vienes de SQL, lo aprendes en una tarde.

Caso Aurora · telemetría de surtidores

- 250 estaciones × 6 surtidores · evento cada 30 s.
- Eventstream desde Event Hub (en aula simulamos con **Sample data**).
- Tabla `TelemetriaSurtidor` con `Timestamp`, `EstacionId`, `SurtidorId`, `Producto`, `Caudal`, `Temperatura`, `Estado`.
- Dashboard · eventos/min, caudal medio, top alertas, mapa.
- **Activator** · si `Temperatura > 65 °C` durante > 3 min, avisar al jefe de estación.

Demo en vivo · 10 min

1. Crear `eh_aurora_telemetria` (Eventhouse).
2. Crear **Eventstream** + **Sample data** → **Bicycle rentals**.
3. **Destination** → **Eventhouse**, mapear a `TelemetriaSurtidor`.
4. Esperar 30 s y abrir **KQL Queryset**.
5. Lanzar las 3 queries adaptadas al esquema generado.
6. Crear un **Real-Time Dashboard** con un par de tiles.
7. Crear un **Activator** simple sobre `Caudal`.

Mensajes clave

- **KQL no asusta** · para un SQL-ero se aprende en una tarde.
- Eventhouse **no reemplaza** al Warehouse: convive con él.
- **Activator** cierra el círculo · del dato al **acto**.

M7 · 25 min

Power BI sobre Fabric

Modelo semántico Direct Lake y publicación

Tipos de modelo semántico

- **Default semantic model** del Lakehouse → tablas detectadas automáticamente, sin relaciones.
- **Custom semantic model** → recomendado para producción:
 - Relaciones, jerarquías, medidas DAX, perspectivas, **RLS**.
- Se crea desde el ítem Lakehouse/Warehouse o desde **Power BI Desktop** apuntando a Fabric.

DAX típico para Aurora



```

Importe Total = SUM(fact_ventas[importe])

Litros Vendidos = SUM(fact_ventas[cantidad])

Importe Año Anterior =
CALCULATE([Importe Total], SAMEPERIODLASTYEAR(dim_tiempo[fecha]))

Variación % =
DIVIDE([Importe Total] - [Importe Año Anterior], [Importe Año Anterior])

Top 5 Estaciones =
CALCULATE([Importe Total], TOPN(5, dim_estacion, [Importe Total]))

```

Publicación, apps y subscriptions

- Reportes web directos en Fabric, o publicados desde Power BI Desktop.
- **Apps** · empaquetan reportes para colectivos (lectores Free necesitan F64+).
- **Subscriptions** programadas, **data alerts**, **comments**.
- **OneLake hub** del workspace · catálogo de items por workspace.

PBIP y Git

- Power BI Desktop guarda en formato **.pbip** (carpeta + JSON / TMDL / PBIR).
- Workspaces de Fabric con **Source control** integrado (Azure DevOps o GitHub).
- Permite **revisar PRs** sobre el modelo semántico y los reportes.

En 2026 ya no hay excusa: el modelo y el reporte son **código versionable**.

Direct Lake · buenas prácticas

- Tablas Delta con **tipos correctos** (no `STRING` para fechas).
- **Particionar** columnas grandes ayuda al **column store**.
- Evitar **calculated columns** complejas en el modelo · llévalas al Lakehouse / Warehouse.
- Vigilar el **fallback to DirectQuery** en el indicador del modelo.

Demo en vivo · 12 min

1. Desde `wh_aurora` → **Nuevo modelo semántico** → 5 tablas + vistas.
2. Renombrar a `sm_aurora_ventas`. Crear relaciones (estrella).
3. Añadir las medidas DAX.
4. + **Nuevo reporte** · KPI **Importe Total**, barras por categoría, mapa por provincia, tabla top 10 clientes.
5. Publicar y mostrar el item en el workspace.
6. Mostrar **App** vacía y cómo se publicaría.
7. Mostrar **Source control** del workspace conectado a un repo Git de demo.

Cierre · todo lo de hoy en una imagen



CSV/Excel → Dataflow Gen2 → Lakehouse (bronze)



Notebook Spark



Lakehouse (silver)



Stored Proc Warehouse



Warehouse (gold)



Direct Lake → Semantic Model



Reporte Power BI

(en paralelo)

Event Hub → Eventstream → Eventhouse → KQL Dashboard / Activator

Deberes para casa antes de la Jornada 2

- Completa los **ejercicios** de `ejercicios/jornada-1/`.
- Tu workspace `aurora-curso-fabric` debe tener:
 - Lakehouse + Warehouse poblados.
 - Notebook ejecutado.
 - Pipeline programado.
 - Reporte Power BI publicado.
- Lectura ligera del índice de `ejercicios/jornada-2/` para llegar contextualizado a **Purview y Fabric IQ**.

¡Hasta la Jornada 2!

Purview · Fabric IQ · gobierno y agentes de datos

Si te atascas, abre un issue en el repo del curso o pásate por el canal de Teams del curso.